

Solving Games Using Artificial Intelligence

Richard Drummond
CSCI 499

Contents

Statement of Purpose

Research / Background

Hardware/Software Required for project

Final project screenshots/explanation

Test Plan

Test Results

Challenges Overcome

Future Enhancements

Final defense presentation slides

Introduction

Description of Project:

My project focuses on using Artificial Intelligence along with a Genetic algorithm in order to evolve to play the games Flappy Bird and Snake. The games used are not the original versions; however, they have been coded to replicate their mechanics and playstyles. The "brain" of the program will evolve over time and make educated decisions in order to progress itself through these games. These decisions are based on a reward system as well as associated weights and biases within the neural network. The AI is rewarded for its progression and punished for mistakes.

Motivation and Statement of Purpose:

As technology progresses, we are continuously gathering data and it is essential to analyze and utilize this data efficiently. Manually interpreting data is costly and inefficient. By using machine learning and artificial intelligence, we can train computers to automatically learn from data and make decisions without human interaction. The AI in my project begins with no base knowledge of how the game is to be played and through trial and error traits are developed that allow it to reach an acceptable conclusion.

Machine learning and artificial intelligence are being used all around the world, from businesses and marketing to self-driving cars. I was motivated to use my interest in video games to gain insight into these fields. Implementing this project allowed me to strengthen my knowledge of Python and forced me to learn more about neural networks and their design. These fields can often seem intimidating and I hope that my implementation helps to influence others to learn more about them.

Research and Background

I began this project with little experience using Python and I attempted to use an open source platform for machine learning called TensorFlow. I made this decision without any prior knowledge of Artificial Intelligence or Neural Networks. I would equate this decision to trying to run a marathon when I didn't even know how to walk yet. The following topics contain various write-ups, tutorials, and lecture videos that helped guide me to finding a solution for my project.

Python & Pygame:

- <https://www.tutorialspoint.com/python/>
- <https://www.pygame.org/wiki/tutorials>

AI & Neural Networks:

- Josh Gordon – Getting started with TensorFlow and Deep Learning
<https://www.youtube.com/watch?v=tYYVSEHq-io>
- Using a Neural Network to solve the OpenAI's CartPole
<https://pythonprogramming.net/openai-cartpole-neural-network-example-machine-learning-tutorial/>
- How to build your own Neural Network from scratch in Python
<https://towardsdatascience.com/how-to-build-your-own-neural-network-from-scratch-in-python-68998a08e4f6>
- Jabrils – Writing My First Machine Learning Game
<https://www.youtube.com/watch?v=ZX2Hyu5WoFg>

Research and Background (continued)

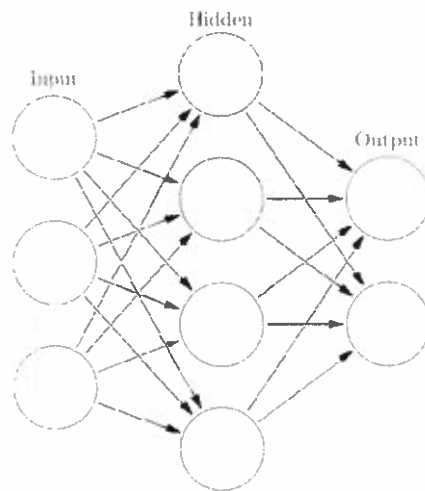
AI & Neural Networks:

- Python Machine Learning Tutorial
https://www.python-course.eu/neural_networks_with_python_numpy.php
- Activation Functions – Relu vs Sigmoid
<https://www.edyoda.com/watch/F356A58F2BBE77FBA7210FC0035D94563>
- Gradient Descent vs. Neuroevolution
<https://towardsdatascience.com/gradient-descent-vs-neuroevolution-f907dace010f>

Genetic Algorithms:

- Siraj Rival - Modeling Evolution with Tensorflow.js
https://www.youtube.com/watch?v=HT1_BHA3ecY
- The Coding Train – Genetic Algorithm Playlist
<https://www.youtube.com/watch?v=9zfeTw-uFCw&list=PLRqwX-V7Uu6bJM3VgzjNV5YxVxUwzALHV>
- Genetic Algorithms Parent Selection
https://www.tutorialspoint.com/genetic_algorithms/genetic_algorithms_parent_selection.htm
- Selection Methods for Genetic Algorithms
https://www.researchgate.net/publication/259461147_Selection_Methods_for_Genetic_Algorithms

Neural Network



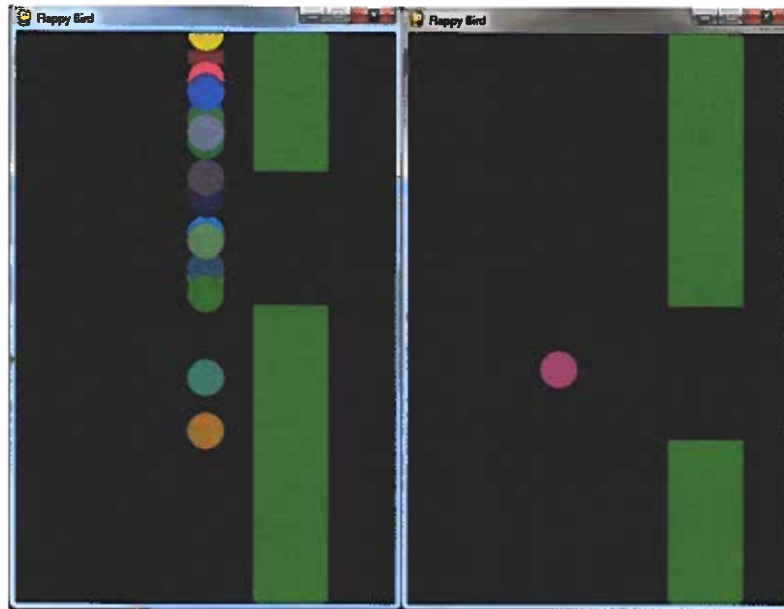
Neural networks are designed solve problems similar to how our brains work. They are made of connected "neurons" that are used to perform calculations. I used a simple artificial neural network from Bernd Klein's Python Machine Learning Tutorial that consists of an Input layer, Hidden layer, and Output layer. As pictured above, the neurons(circles) are connected by weights (arrows). These weights are in the form of a matrix and are values between -1 and 1. When a prediction is being made, the network performs matrix multiplication using the weights and input. The output is determined using sigmoid as an activation function and will contain values between 0 and 1.

Software Requirements

Python3 is required as well as the following libraries:

- Pygame – game creation
- NumPy – scientific computing package
- SciPy – open-source software for mathematics, science, and engineering
- Copy – allows for deep/shallow copy operations
- Pickle – protocols for serializing and de-serializing Python objects

Flappy Bird



Flappy Bird is a side scrolling game where the player controls a bird (the colored circles) and tries to navigate between pipes (the green rectangles). The only input from the player is to flap their wings to increase their height, else they will fall downward. The game ends if the player collides with the ceiling, floor, or a pipe. The players score is determined by how long they survived and the amount of pipes they flew through.

Pictured from left to right, a population of 100 birds are generated and eventually there is only one survivor. Each bird has their own neural network that determines whether or not the bird should "flap". The neural networks are based on 5 input nodes, 5 hidden nodes, and 1 output node. The input consists of the birds height and current change in height (+/- velocity) as well as information on the closest incoming pipe (position, top of gap, bottom of gap). After processing the information the network will output a number between 0 and 1. If the output is greater than .5 the determined action is to flap and increase its height. These decisions continue until every bird has failed.

Snake



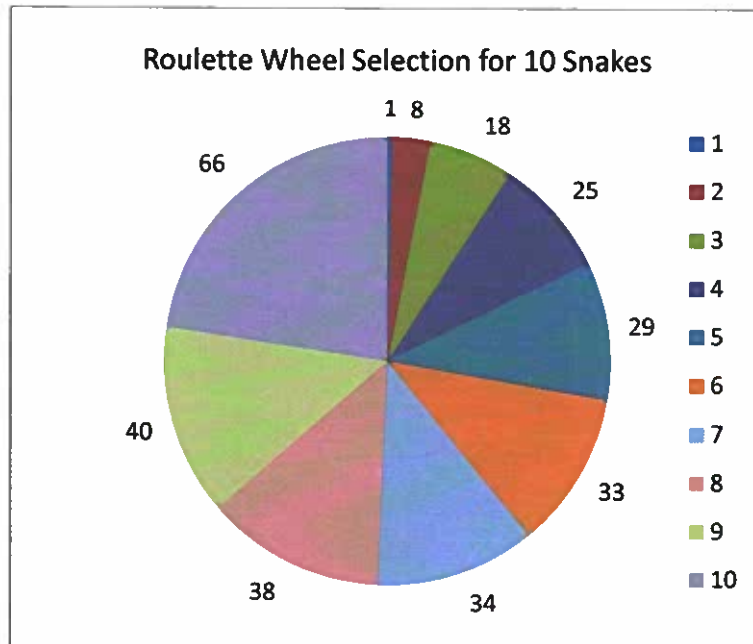
Snake is a game where the player controls a snake (pictured in green) that grows in length whenever it 'eats' food (the red square). The player controls the snake using four inputs of up, down, left, or right. The snake continually moves in a given direction until changed again by the player and is not allowed to move in the opposite direction it is currently traveling. The game ends if the player collides with itself or a wall. The player's score is determined by the amount of food that is eaten.

A population of 100 snakes are generated and play consecutively. Each snake has its own neural network that determines if it should continue in the same direction or turn. The neural networks are based on 6 input nodes, 4 hidden nodes, and 3 output nodes. Each input contains a 1 or 0 that corresponds with whether or not the left, forward, or right direction will result in a collision or if that direction contains the food. After processing the information, the network produces three outputs that correspond to choosing left, forward, or right. The largest number of the three outputs is chosen to be the action the snake will take.

Since Snake's population trains consecutively I added quality of life improvements. Pressing the up/down arrows will increase/decrease the speed at which the game is played. Pressing the left/right arrow keys will toggle the game to play at an increased speed without rendering the graphics. This drastically reduced training times.

Genetic Algorithm

This algorithm works the same for each game, I will proceed by referencing Snake. When the final snake fails, a new population is generated based on the scores of the previous snakes. A roulette wheel selection is used to make it more likely that snakes with a higher score will be chosen to produce the next population. After each selection is made, the snakes neural network is cloned and given a chance to mutate. A mutation rate of 1 would mean that each snake is completely randomized and a rate of 0 would mean an exact clone. Assuming the mutation rate is 10%, each weight in the neural network has a 10% chance to shift by a standard deviation. This mutation is done to ensure variation with each new population.



Population Selection

```
# roulette wheel selection based on each snakes score
def getSnake(self, total):
    choice = None
    select = random.randrange(0, total)
    partial = 0
    for snake in self.oldSnakes:
        partial += snake.score
        if partial > select:
            choice = snake
            break
    newSnake = Snake(choice.nn)
    newSnake.nn.mutate(MRATE)
    return newSnake
```

This function takes the input 'total' which is the sum of the current generations scores. A random number between 0 and the total is chosen and is the target for our selection. We then iterate through the population and store the sum of each snakes score. When the sum of the snakes scores is greater than our target (partial > select) the current snake is our choice for reproduction. We then create a new snake using the old snakes neural network. Lastly, the new snake is given a chance to mutate.

Item Pass/Fail Criteria:

- Snake.py will fail based on the following:
 - Snake does not move after receiving directional input
 - Food is not generated (initially or after collision with the snake)
 - Food spawns within the snakes body
 - Collision is not detected between the snake and itself, food, or walls
 - Score does not update
 - Game does not close when escape key is pressed
- neuralNetwork.py will fail based on the following:
 - Neural Network is not generated upon receiving input
 - Neural Network is unable to return output
 - Weights do not change as expected with given mutation rate
- SnakeGA.py will fail based on the following:
 - Directional input not detected from AI
 - Decisions lead to infinite loop for survival
 - Decisions consistently lead to collision with self or walls
 - AI does not collide with the food at least 10 times
 - Score does not increase over time / amount of games played
 - Game Speed is unaffected by user input
 - AI does not continue to train when graphics are not rendered

Suspension Criteria and Resumption Requirements:

- Testing will be suspended if the AI continues to fail game mechanics.
- Testing will proceed after the AI's decision making process has been reviewed.

Test Deliverables:

- Test Report
- Graph of median and max. scores

Test Environment:

- Python version 3.6.1
- Pygame
- NumPy

Training Needs:

- Knowledge of Python and varying libraries.

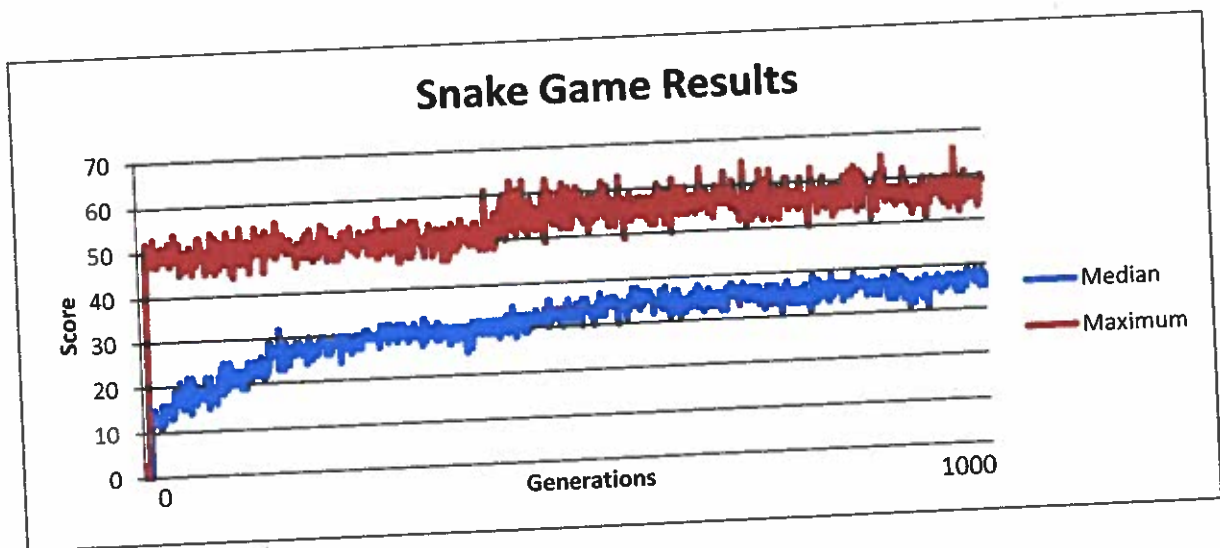
Approvals:

- Dr. Sean Hayes

Test Results

My base game, Snake.py, failed initially due to the food being able to spawn within the body of the snake. The neural network failed during most of my testing because I initially did a shallow copy rather than a deep copy when making my selections. The AI failed repeatedly as I experimented with changes in variables such as mutation rate, population size, and neural network inputs.

The base game features and functionality are now working as intended. The neural network is able to be generated, copied, and mutated. The AI, while not fully optimized, is able to surpass the initial test case by a large margin. Initially, I decided that the AI should be able to collide with the food at least 10 times without dying (score = 10). As pictured below, the maximum score after 1000 generations is 66 and the median score peaked at 39.



Challenges

My biggest challenge starting this project was that I wasn't experienced with Python and I was trying to use open source tools that I did not understand. I struggled trying to implement a neural network without knowing the language or the theories behind what I was doing. After repeatedly trying and failing, I began to do more research in order to find a solution. I found tutorials, YouTube videos, and various write-ups that eventually led me to finding a more suitable way to solve my problem.

Having previously studied Biology, I was intrigued when I came across the idea of using a genetic algorithm with a neural network. This immediately seemed more approachable for me and I began to focus on using this solution with Flappy Bird. Following Flappy Bird, I felt that I had a much better understanding and went back to finding a solution for Snake.

Aside from technical challenges, there were mistakes that could have been eliminated earlier if I hadn't assumed the code I wrote worked correctly. I made small mistakes such as not resetting a 1 back to a 0 or I was using the wrong variable. In the grand scheme of things, most of my challenges could have been avoided if I did my research before getting started and I made more time to practice what I was doing.

Future Enhancements

Optimization:

Each of these games neural networks could be optimized by continuing to manipulate the inputs and hidden nodes until better results are achieved. Exact rules do not exist for determining these variables other than testing them to fit your required solution. Further experimentation is required to find an optimal solution to each problem.

Cross-over Function:

A cross-over function ensures that increased variation occurs with each population. This function would pair with our selection and mutation process. Instead of cloning a neural network, we would choose two 'parents' and use weights from each of them to generate a 'child' neural network. This child would still be given a chance to mutate before being added to the new population.

Convolutional Neural Network (CNN):

This type of neural network would be a different approach than using a genetic algorithm. I intend to use a machine learning platform such as TensorFlow or PyTorch in order to solve these games using a convolutional neural network. CNN's are able to use image data as an input and predict an output using learnable weights and biases (Saha, Sumit). In theory, the neural network will train itself over time by analyzing the games image data frame by frame and detecting patterns.

Solving Games Using Artificial Intelligence

Richard Drummond
Major: Computer Science

Why?

- Interest in games
- New experience
- Problem solving

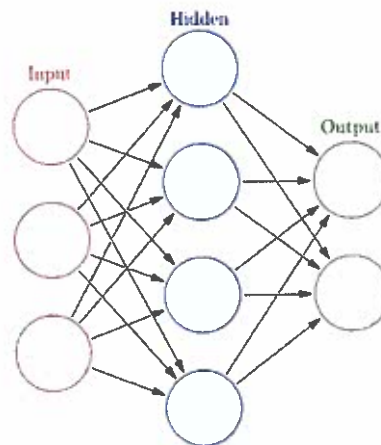
How?

- Neural Network
- Genetic Algorithm
- Time

3

Neural Network

- Layers of “neurons”
- Weights and Biases
- Linear Algebra



4

Genetic Algorithm

- Survival of the fittest
 - Initial Population
 - Fitness Score
 - Reproduction
 - Mutation

5

Implementation

- Python
 - Pygame
 - NumPy
 - SciPy
- Games:
 - Flappy Bird
 - Snake

6

Demo

7

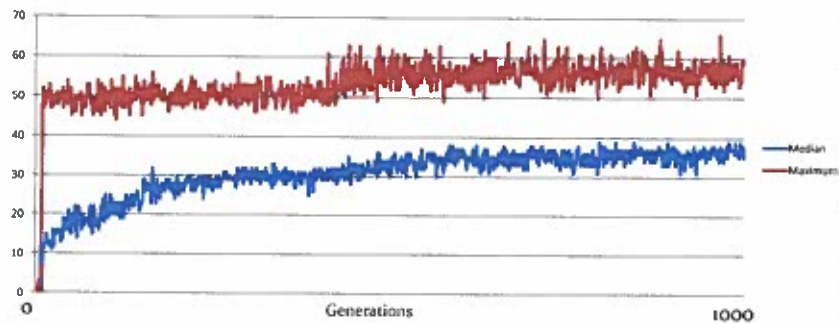
Testing

- Manual testing of game functionality
- Statistical Analysis
 - Input/Hidden Layers
 - Population Size
 - Fitness Selection
 - Mutation Rate

8

Snake Game Results

- Population: 100,000
- Best Score: 66



9

Challenges

- Changing projects
- Python
 - Lack of experience, unfamiliar with syntax
- TensorFlow
 - Can't swim? Jump in the deep end!

Solution: research, practice, and time management

10

Future Enhancements

- Optimize Input/Hidden Layers
- Crossover function
 - Two parent selection for offspring
- Deep Learning
 - Convolutional Neural Network
 - Machine learning platform such as Pytorch or TensorFlow

11

Questions?

12